
Tutorial Sheet – Abstraction and Polymorphism

Programming Fundamentals 2

Topic: Abstraction, Abstract Classes and Polymorphism

Purpose: Understanding inheritance hierarchies and polymorphic behaviour in Java

Part A – Understanding Abstraction

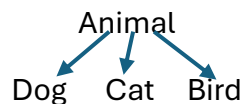
1. Abstract vs Concrete Classes

Answer the following questions.

- What is an **abstract class** in Java?
 - Can an abstract class be instantiated? Explain why.
 - What is an **abstract method**?
 - What must a subclass do if it inherits an abstract method?
-

2. Identifying Abstract Classes

Consider the following class hierarchy for a **zoo system**:



Answer the following:

- Why might Animal be designed as an **abstract class**?
 - Give **two methods** that might reasonably exist in the Animal class.
 - Suggest **one abstract method** that each subclass would need to implement.
-

Part B – Writing Abstract Classes

3. Creating an Abstract Class

Write a Java class called Shape with the following:

- A protected field name
- A constructor that sets name
- An **abstract method** called calculateArea()

4. Implementing a Subclass

Write a subclass called Circle that:

- Extends Shape
- Has a field radius
- Implements the calculateArea() method

Use the formula:

$$\text{area} = \pi \times \text{radius} \times \text{radius}$$

Part C – Understanding Polymorphism

5. Static Type vs Dynamic Type

Consider the following code:

```
Vehicle v1 = new Car();
```

Answer the following:

- a) What is the **static type** of v1?
 - b) What is the **dynamic type** of v1?
 - c) Which type is checked by the **compiler**?
 - d) Which type determines which method runs at runtime?
-

6. Predicting Method Behaviour

Consider the following classes:

```
class Animal {  
    public void makeSound() {  
        System.out.println("Animal sound");  
    }  
}
```



```
class Dog extends Animal {  
    public void makeSound() {  
        System.out.println("Bark");  
    }  
}
```

And the following code:

```
Animal a = new Dog();  
a.makeSound();
```

What will be printed?

Explain **why** this happens.

Part D – Overriding Methods

7. Method Overriding

Answer the following questions.

- a) What does **method overriding** mean?
 - b) When overriding a method, what must remain the same?
 - c) Why is overriding important for polymorphism?
-

8. Using super

Consider the following code:

```
class Post {  
    public String display() {  
        return "Post information";  
    }  
}  
  
class MessagePost extends Post {  
    public String display() {
```

```
        return super.display() + " - Message content";
    }
}
```

Answer the following:

- a) What does the keyword super do?
- b) What will the following print?

```
MessagePost m = new MessagePost();
System.out.println(m.display());
```

Part E – Polymorphism in Collections

9. Polymorphic ArrayList

Consider the following code:

```
ArrayList<Post> posts = new ArrayList<Post>();
```

```
posts.add(new MessagePost("Alice"));
posts.add(new PhotoPost("Bob"));
posts.add(new EventPost("Charlie"));
```

Answer the following:

- a) Why can the ArrayList store different types of objects?
 - b) What do all these objects have in common?
 - c) What concept in Java allows this behaviour?
-

Part F – The instanceof Operator

10. Identifying Object Types

Consider the following code:

```
if(post instanceof MessagePost){  
    MessagePost msg = (MessagePost) post;  
}
```

Answer the following:

- a) What does the instanceof operator do?
 - b) Why is the cast (MessagePost) necessary?
 - c) When might using instanceof be useful?
-

Part G – Design Question

11. Designing with Abstraction

You are designing a **Payment System**.

Possible payment types include:

- CreditCardPayment
- PayPalPayment
- BankTransferPayment

Answer the following:

- a) What class should act as the **abstract superclass**?
 - b) Suggest **two fields** that should belong in this superclass.
 - c) Suggest **one abstract method** that all subclasses should implement.
 - d) Explain why abstraction is useful in this situation.
-

Part H – Thinking Question

12. Why Abstraction Matters

In the **Social Network example**, we never create objects of type `Post`, only:

- `MessagePost`
- `PhotoPost`
- `EventPost`

Explain **two reasons** why `Post` should be an **abstract class**.